

Numbers – Simple Hinglish Summary

Source: <https://javascript.info/number>

Main baat

Modern JavaScript me **do types ke numbers** hain:

1. **Regular numbers:** 64-bit format **IEEE-754** me store hote hain ("double precision floating point"). Hum zyadatar yahi use karte hain. Is chapter me inhi ki baat hai.
2. **BigInt:** kisi bhi length ke integers. Regular number $\pm(2^{53} - 1)$ se bade integers safely store nahi kar sakta, isliye kabhi kabhi BigInt chahiye. Iska alag chapter hai.

1. Number likhne ke aur tarike

1 billion likhna ho:

```
javascript  
  
let billion = 1000000000;  
let billion = 1_000_000_000; // underscore se readable
```

Underscore `_` sirf **syntactic sugar** hai (readability ke liye). Engine use ignore kar deta hai.

Asli zindagi me bahut saare zero likhne ki jagah `e` use karte hain:

```
javascript  
  
let billion = 1e9; // 1 billion (1 aur 9 zeroes)  
  
alert( 7.3e9 ); // 7.3 billion (7300000000)
```

`e` number ko `1` ke saath diye gaye zeroes se multiply karta hai:

```
javascript  
  
1e3 === 1 * 1000;  
1.23e6 === 1.23 * 1000000;
```

Bahut chhota number (1 microsecond):

```
javascript  
  
let mcs = 1e-6; // 0.000001
```

e) ke baad negative number ka matlab hai divide karna:

javascript

```
1e-3 === 1 / 1000;           // 0.001
1.23e-6 === 1.23 / 1000000; // 0.00000123
1234e-2 === 1234 / 100;      // 12.34
```

Hex, binary aur octal numbers

- **Hex:** `(0x)` se shuru. Colors, character encoding, etc. me use hota hai.

javascript

```
alert( 0xff ); // 255
alert( 0xFF ); // 255 (case matter nahi karta)
```

- **Binary:** `(0b)`, **Octal:** `(0o)` (kam use hote hain):

javascript

```
let a = 0b11111111; // 255 ka binary
let b = 0o377;       // 255 ka octal

alert( a == b ); // true
```

Sirf yehi 3 systems direct supported hain. Baaki ke liye `parseInt` use karo.

2. `toString(base)`

`num.toString(base)` number ko given numeral system me **string** banakar deta hai.

javascript

```
let num = 255;

alert( num.toString(16) ); // ff
alert( num.toString(2) );  // 11111111
```

`base` 2 se 36 tak ho sakta hai. Default 10.

Common use:

- **base=16:** hex colors, character encodings (digits `(0..9)`, `(A..F)`)
- **base=2:** bitwise operations debug karna

- **base=36:** maximum. Poore Latin alphabet ke saath. Lambe numeric ID ko chhota banane ke liye (jaise short URL):

```
javascript
```

```
alert( 123456..toString(36) ); // 2n9c
```

Do dots kyu? Number par seedha method call karna ho to `(..)` (do dots) likhna padta hai. Ek dot par JS decimal part samajhta hai aur error aata hai. Do dots par JS samajhta hai ki decimal part khali hai. Ya `((123456).toString(36))` bhi likh sakte ho.

3. Rounding

Sabse zyada use hone wala operation. Kai built-in functions hain:

- **Math.floor:** neech round karta hai. `(3.1) → (3)`, `(-1.1) → (-2)`
- **Math.ceil:** upar round karta hai. `(3.1) → (4)`, `(-1.1) → (-1)`
- **Math.round:** nazdeeki integer tak. `(3.1) → (3)`, `(3.6) → (4)`. Beech wale cases me `(3.5) → (4)`, `(-3.5) → (-3)`
- **Math.trunc:** decimal ke baad ka sab hata deta hai (bina round kiye). `(3.1) → (3)`, `(-1.1) → (-1)`

	floor	ceil	round	trunc
3.1	3	4	3	3
3.5	3	4	4	3
3.6	3	4	4	3
-1.1	-2	-1	-1	-1
-1.5	-2	-1	-1	-1
-1.6	-2	-1	-2	-1

Decimal ke `(n)`-th digit tak round karna

`(1.2345)` ko 2 digit tak round karke `(1.23)` chahiye. Do tarike:

1. Multiply-and-divide:

```
javascript
```

```
let num = 1.23456;

alert( Math.round(num * 100) / 100 ); // 1.23
```

2. `toFixed(n)`: `(n)` digits tak round karta hai aur **string** return karta hai:

```
javascript

let num = 12.34;
alert( num.toFixed(1) ); // "12.3"

let num2 = 12.36;
alert( num2.toFixed(1) ); // "12.4"
```

Decimal part chhota ho to end me zeroes jod deta hai:

```
javascript

let num = 12.34;
alert( num.toFixed(5) ); // "12.34000"
```

Number me badalna ho to unary plus ya `Number()`: `+num.toFixed(5)`.

4. Imprecise calculations (galat-sa hisaab)

Number 64 bits me store hota hai: **52 bits digits ke liye, 11 bits decimal point ki position ke liye, 1 bit sign ke liye.**

Number bahut bada ho to `Infinity` ban jata hai:

```
javascript

alert( 1e500 ); // Infinity
```

Precision ka nuksan (bahut common):

```
javascript

alert( 0.1 + 0.2 == 0.3 ); // false
alert( 0.1 + 0.2 ); // 0.30000000000000004
```

Sochlo e-shop me `($0.10)` aur `($0.20)` ke items ka total `($0.30000000000000004)` dikhe to?

Ye kyu hota hai? Number memory me **binary** me store hota hai. `(0.1)`, `(0.2)` jaise fractions decimal me simple dikhte hain, lekin binary me **kabhi khatam na hone wale fractions** hote

hain (jaise decimal me $1/3 = 0.333\dots$).

0.1 **exactly** binary me store karna namumkin hai, bilkul waise hi jaise ek-tihai ko decimal fraction me exactly likh nahi sakte. IEEE-754 format nazdeeki possible number tak round karta hai, isliye ye chhota nuksan aam taur par dikhta nahi. Lekin do numbers add karne par ye nuksan jud jate hain.

javascript

```
alert( 0.1.toFixed(20) ); // 0.10000000000000000555
```

Sirf JS me nahi: PHP, Java, C, Perl, Ruby me bhi yahi hota hai.

Workaround: `toFixed(n)` se round karo:

javascript

```
let sum = 0.1 + 0.2;
alert( sum.toFixed(2) ); // "0.30"
```

`toFixed` hamesha **string** deta hai. Number chahiye to unary plus: `+sum.toFixed(2)` → `0.3`.

Ya 100 se multiply karke integer bana lo, maths karo, wapas divide karo (error kam hota hai lekin पूरी तरह khatam nahi):

javascript

```
alert( (0.1 * 10 + 0.2 * 10) / 10 ); // 0.3
alert( (0.28 * 100 + 0.14 * 100) / 100 ); // 0.42000000000000001
```

Fractions se bachne ka bhi tarika hai (jaise prices cents me rakho), lekin discount (30%) laga do to phir fractions aa jate hain. Aam taur par bas jab zarurat ho, round kar do.

Mazedar baat:

javascript

```
alert( 9999999999999999 ); // 10000000000000000000 dikhata hai
```

Yahan bhi precision ka nuksan hai. 52 bits digits ke liye kaafi nahi, isliye sabse chhote digits gayab ho jate hain. JS koi error nahi deta.

Do zero: `0` aur `-0` dono hote hain, kyunki sign ek alag bit hai. Zyadatar cases me farak nahi dikhta.

5. `isFinite` aur `isNaN`

Do special values: `Infinity` (aur `-Infinity`) aur `NaN` (error). Ye `number` type ke hain lekin "normal" numbers nahi, isliye inke liye special functions hain.

`isNaN(value)`: argument ko number me convert karke check karta hai ki wo `NaN` hai ya nahi:

javascript

```
alert( isNaN(NaN) );    // true
alert( isNaN("str") );  // true
```

`NaN === NaN` se kyu nahi? Kyunki `NaN` kisi ke barabar nahi hota, khud ke bhi nahi:

javascript

```
alert( NaN === NaN );  // false
```

`isFinite(value)`: number me convert karke `true` deta hai agar wo regular number hai (`NaN/Infinity/-Infinity` nahi):

javascript

```
alert( isFinite("15") );    // true
alert( isFinite("str") );    // false (NaN)
alert( isFinite(Infinity) ); // false
```

Kabhi kabhi string valid number hai ya nahi, ye check karne ke liye use hota hai:

javascript

```
let num = +prompt("Enter a number", '');

// true hoga jab tak Infinity, -Infinity ya number nahi daala
alert( isFinite(num) );
```

Dhyan: Khali ya sirf space wali string saare numeric functions me `0` maani jati hai (`isFinite` me bhi).

`Number.isNaN` aur `Number.isFinite` (strict versions)

Ye argument ko **number me convert nahi karte**, balki check karte hain ki wo `number` type ka hai ya nahi.

javascript

```
alert( Number.isNaN(NaN) );           // true
alert( Number.isNaN("str" / 2) );     // true

// Farak dekho:
alert( Number.isNaN("str") ); // false ("str" string type ka hai)
alert( isNaN("str") );        // true (convert karke NaN mila)

alert( Number.isFinite(123) );        // true
alert( Number.isFinite("123") );      // false (string type)
alert( isFinite("123") );             // true (convert karke 123 mila)
```

Practice me zyadatar `isNaN` aur `isFinite` hi use hote hain kyunki wo chhote hain.

`Object.is`

`===` jaisa compare karta hai, lekin do edge cases me zyada reliable:

1. `NaN` ke saath kaam karta hai: `Object.is(NaN, NaN) === true`
2. `0` aur `-0` alag hain: `Object.is(0, -0) === false`

Baaki sab cases me `a === b` jaisa hi. Ye specification me aksar use hota hai.

6. `parseInt` aur `parseFloat`

`+` ya `Number()` se numeric conversion **strict** hota hai. Value exactly number na ho to fail:

javascript

```
alert( +"100px" ); // NaN
```

(Sirf shuru/end ke spaces ignore hote hain.)

Lekin real life me `"100px"`, `"12pt"`, `"19€"` jaisi values aati hain. Iske liye `parseInt` aur `parseFloat`:

Ye string se number tab tak "padhte" hain jab tak padh sakein. Error aane par jitna padha wo return karte hain. `parseInt` integer, `parseFloat` floating-point return karta hai:

javascript

```
alert( parseInt('100px') );    // 100
alert( parseFloat('12.5em') ); // 12.5

alert( parseInt('12.3') );    // 12 (sirf integer part)
alert( parseFloat('12.3.4') ); // 12.3 (doosra point rok deta hai)
```

Koi digit na padh paayein to **NaN**:

```
javascript

alert( parseInt('a123') ); // NaN
```

parseInt(str, radix) ka dusra argument: numeral system ka base batata hai:

```
javascript

alert( parseInt('0xff', 16) ); // 255
alert( parseInt('ff', 16) );   // 255 (0x ke bina bhi chalta hai)

alert( parseInt('2n9c', 36) ); // 123456
```

7. Aur math functions

JS me built-in **Math** object hai jisme chhoti si library hai.

- **Math.random()**: 0 se 1 ke beech random number (1 shamil nahi)

```
javascript

alert( Math.random() ); // 0.1234567894322
```

- **Math.max(a, b, c...)** aur **Math.min(a, b, c...)**: sabse bada aur sabse chhota

```
javascript

alert( Math.max(3, 5, -10, 0, 1) ); // 5
alert( Math.min(1, 2) );             // 1
```

- **Math.pow(n, power)**: **n** ki power

```
javascript

alert( Math.pow(2, 10) ); // 1024
```

Aur functions (trigonometry, etc.) `Math` docs me hain.

Summary

Bahut zeroes wale numbers:

- `123e6` = `123000000`
- `123e-6` = `0.000123`

Alag numeral systems:

- Seedha hex (`0x`), octal (`0o`), binary (`0b`) likh sakte hain.
- `parseInt(str, base)`: string ko given base ke integer me parse karta hai (`2 ≤ base ≤ 36`).
- `num.toString(base)`: number ko given base ki string banata hai.

Regular number tests:

- `isNaN(value)`: number me convert karke `NaN` check
- `Number.isNaN(value)`: `number` type ho tabhi `NaN` check
- `isFinite(value)`: convert karke `NaN/Infinity/-Infinity` na hone ka check
- `Number.isFinite(value)`: `number` type ho tabhi wahi check

`12pt`, `100px` jaisi values ke liye: `parseInt/parseFloat` ("soft" conversion).

Fractions ke liye: `Math.floor`, `Math.ceil`, `Math.trunc`, `Math.round` ya

`num.toFixed(precision)` se round karo. Yaad rakho ki fractions me precision ka nuksan hota hai.

Practice Tasks (Answers)

1. Visitor se numbers ka sum:

```
javascript

let a = +prompt("The first number?", "");
let b = +prompt("The second number?", "");

alert( a + b );
```

`prompt` se pehle unary `+` zaruri hai, warna strings jud jayengi (`"1" + "2" = "12"`).

2. `6.35.toFixed(1) == 6.3` kyu? `6.35` andar se binary me endless fraction hai, aur thoda kam store hota hai:

```
javascript
```

```
alert( 6.35.toFixed(20) ); // 6.34999999999999964473
```

Isliye neeche round hua. `1.35` thoda zyada store hota hai (`1.35000000000000008882`), isliye upar round hua.

Sahi round karne ke liye pehle integer ke paas laao:

javascript

```
alert( Math.round(6.35 * 10) / 10 ); // 6.4
```

(`63.5`) me precision ka nuksan nahi, kyunki (`0.5 = 1/2`) binary me exact hai.)

3. `readNumber` (jab tak number na mile poochte raho):

javascript

```
function readNumber() {  
  let num;  
  
  do {  
    num = prompt("Enter a number please?", 0);  
  } while ( !isFinite(num) );  
  
  if (num === null || num === '') return null;  
  
  return +num;  
}  
  
alert(`Read: ${readNumber()}`);
```

(`null`) (cancel) aur khali line dono numeric form me (`0`) hain, isliye (`isFinite`) unhe rokta nahi. Loop ke baad unhe alag se (`null`) return karna padta hai.

4. Ek kabhi khatam na hone wala loop:

javascript

```
let i = 0;  
while (i != 10) {  
  i += 0.2;  
}
```

(i) kabhi exactly 10 nahi hoga (0.2) jodne se precision ka nuksan). **Seekh:** decimal fractions ke saath equality checks se bacho.

5. min se max tak random float:

javascript

```
function random(min, max) {  
  return min + Math.random() * (max - min);  
}
```

6. min se max tak random integer (dono shamil):

Galat tareeka (Math.round se): edge values (min, max) ki probability aadhi ho jati hai.

Sahi tareeka:

javascript

```
function randomInteger(min, max) {  
  // rand (min-0.5) se (max+0.5) tak  
  let rand = min - 0.5 + Math.random() * (max - min + 1);  
  return Math.round(rand);  
}
```

Ya Math.floor se:

javascript

```
function randomInteger(min, max) {  
  // rand min se (max+1) tak  
  let rand = min + Math.random() * (max + 1 - min);  
  return Math.floor(rand);  
}
```

Quick Summary

Kaam	Kaise
Bahut zeroes	<code>1e9</code> , <code>1.23e-6</code> , <code>1_000_000</code>
Hex / binary / octal	<code>0xff</code> , <code>0b11</code> , <code>0o377</code>
Base badalna	<code>num.toString(16)</code> , <code>parseInt("ff", 16)</code>
Neeche / upar round	<code>Math.floor</code> , <code>Math.ceil</code>
Nazdeeki round	<code>Math.round</code>
Decimal kaatna	<code>Math.trunc</code>
<code>n</code> digits tak	<code>num.toFixed(n)</code> (string deta hai)
<code>0.1 + 0.2</code>	<code>0.30000000000000004</code> (round karo)
<code>NaN</code> check	<code>isNaN()</code> (<code>=== NaN</code> nahi chalta)
<code>"100px"</code> se number	<code>parseInt("100px")</code>
Random	<code>Math.random()</code>
Bada/chhota	<code>Math.max()</code> , <code>Math.min()</code>
Decimals ke saath equality	Mat karo (precision ka nuksan)